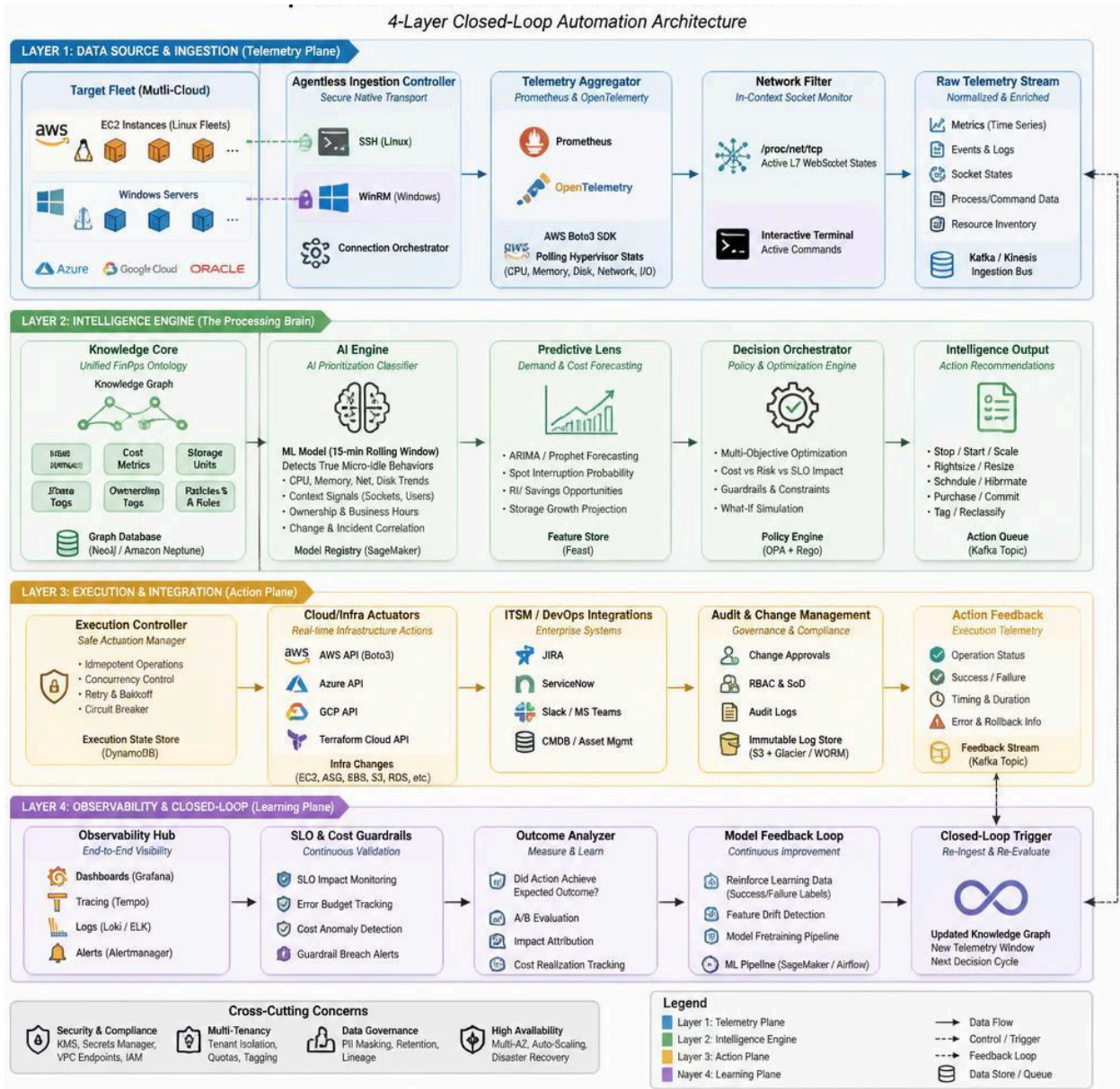
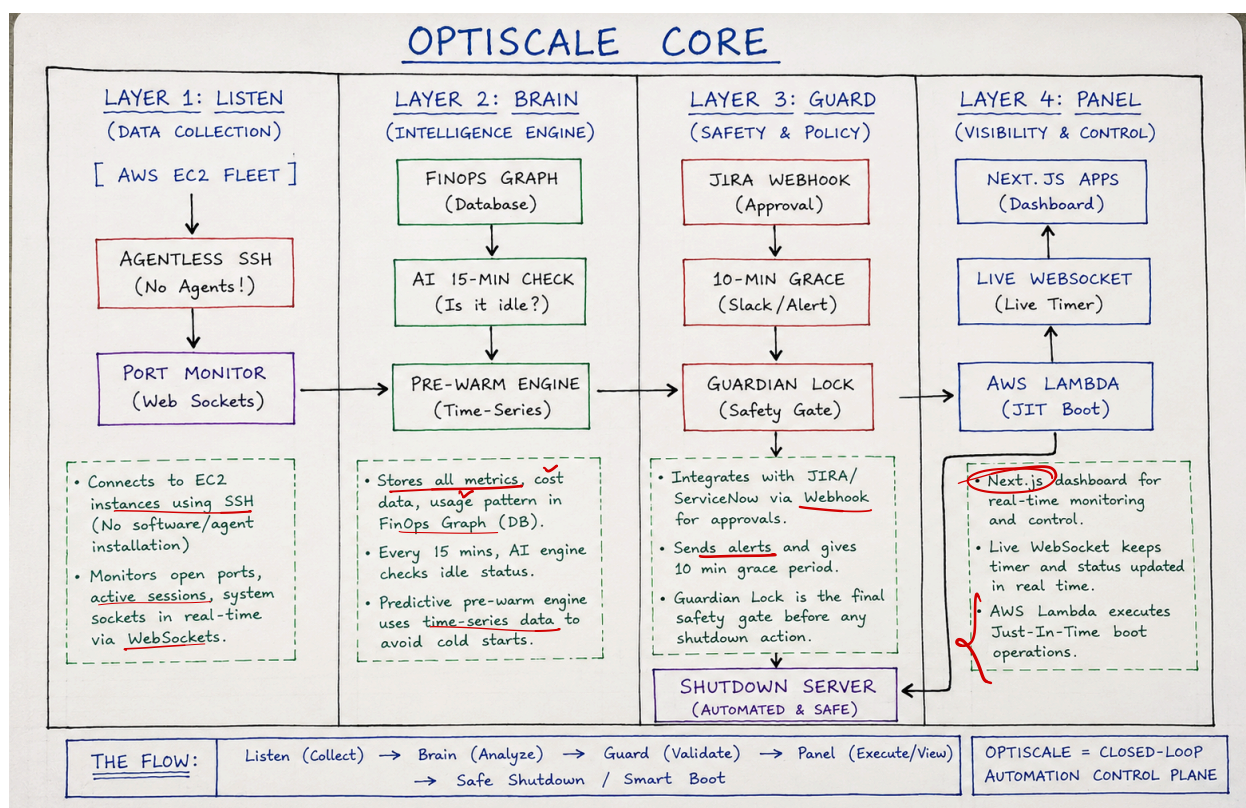


|| Shri Ganesh ||

Optiscale = our platform name

Detailed Architecture





Layer 1 — Data Source & Ingestion (The Telemetry Plane)

What It Does:

This layer forms the ingestion backbone of the framework. It connects directly to the target **cloud virtual machines (EC2 instances)** and underlying operating systems to stream real-time utilization and **financial data without installing** invasive local daemons (Agentless Ingestion).

Components:

- **Agentless OS Telemetry Stream:** A lightweight background engine that uses secure native pipelines (SSH for Linux, WinRM for Windows) to execute low-impact system resource queries.
- **Cloud API Connectors (Boto3 Framework):** Intercepts live cloud hypervisor statistics, pulling metadata regarding instance configurations, current state (Running/Stopped), pricing tier schedules, and active Elastic IP mappings.

→ website

→ python library

- **Unified Metric Pipeline (Prometheus & OpenTelemetry):** Standardizes incoming time-series telemetry streams—capturing CPU percentages, memory allocation footprints, disk I/O, and byte transfer rates.
- **The In-Context Socket Filter:** An operational rule-engine that inspects layer-7 network socket tables (reading /proc/net/tcp or executing `ss` on Windows) to actively count live human connections on web ports (80/443).

Why It Matters:

By leveraging completely agentless ingestion standards, OptiScale can be deployed across existing corporate legacy infrastructure and mixed-OS server fleets instantly with zero code changes or local software installations.

Layer 2 — Intelligence Engine (FinOps Ontology + AI Agents)

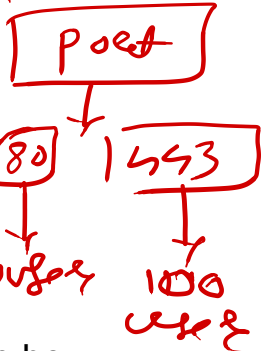
What It Does:

This is the central nervous system of the project. It transforms raw usage numbers and billing metrics into immediate, actionable operational intelligence: determining if a server is genuinely idle, mapping costs to user value, and calculating risk thresholds.

Components:

- **Unified FinOps Ontology (The Infrastructure Knowledge Graph):** A relational data structure (implemented via PostgreSQL or a light graph database) that links Instances, Associated Storage Blocks, Active Developers, Team Budgets, JIRA Ticket History, and Real-Time Billing Rates. This serves as the single source of truth for the system.
- **The AI Prioritization Agent:** A machine learning classifier (inspired by XGBoost) trained on historical time-series logs. Instead of looking at a brief snapshot, it analyzes a rolling 15-minute window to identify "Micro-Idle" patterns, calculating the exact probability that a server is wasting money without running an active work task.

Realtime Data



110 users

like CPU usage - - etc

best for realtime scale Data

→ ML model

we can change according our understanding

- We will change it according to understanding*
- **The Predictive Pre-Warming Engine:** Examines historical system usage trajectories stored in the database. If it identifies that developers or end-users reliably hit the platform at 8:00 AM every morning, it generates an optimized scheduling block to wake the infrastructure ahead of time.
 - **Context & Unit Economics Resolver:** Continuously divides infrastructure spend metrics by incoming application request volume to determine the live **Cost-per-Request.** It identifies which microservices or user tiers are causing financial anomalies.

Layer 3 — Mediation & Configuration (Governance & Safety Controls)

What It Does:

This layer bridges business logic and enterprise security with automated infrastructure execution. It represents the project's unique architectural safety net, ensuring automated optimization rules never compromise system uptime.

Components:

- **Bi-Directional Workflow Bridge (JIRA/ServiceNow Webhooks):** When an optimization threshold is breached, this component halts immediate technical execution and automatically creates structured, policy-driven approval tickets within the team's ticketing system (JIRA).
- **The 10-Minute User Notification Loop:** Manages live communication out to Slack, Teams, or the frontend command panel. It handles the **10-minute User Grace Period** countdown, allowing on-call developers to instantly issue a manual override if critical work is in flight.
- **The Guardian Safety Validator:** A strict rule-checking pipeline that double-checks every automated optimization command right before it hits the cloud APIs. It runs mandatory validation checks: *Are there*

active human SSH terminals? Is an automated database backup running? Is this a mission-critical holiday window?

- **The Fallback Default Engine:** If an automated scale-down recommendation fails the Guardian Validator's safety checks, the system aborts the operation, updates the JIRA ticket status to "Blocked", and falls back to a safe default "Pilot Light" state, ensuring the system remains fail-safe.

Layer 4 — Presentation (The Next-Gen Command Cockpit)

What It Does:

Delivers a high-density, real-time command interface to the right stakeholder on any device, abstracting cloud infrastructure management into a beautiful, collaborative viewport.

Components:

- **The Responsive Next.js Control Panel:** A clean frontend interface optimized for low latency and data-density, utilizing custom backdrop-blurs, real-time state grids, and clear operational badge patterns.
- **WebSocket Live Update Streams:** Establishes a permanent, two-way persistent pipeline between the PostgreSQL/FastAPI backend and the React components, updating active instance statuses, grace-period timers, and ticket state approvals in under a second.
- **The Financial Scoreboard Dashboard:** Provides real-time financial tracking widgets, displaying calculated gross margin metrics, budget cap thresholds, and an active digital "**Prevented Waste Counter**" that increments dynamically.
- **Just-In-Time (JIT) Activation Engine (AWS Lambda "Fast-Boot"):** Features a prominent, accessible user control interface. If a developer needs immediate access to a safely stopped or shrunken instance, clicking this button triggers an immediate AWS Lambda API sequence that wakes the target server in seconds.



Core Keyword Glossary (Term Matrix for the Team)

Technical Term	Simple Explanation	Strategic Value to the Project
Agentless Architecture	Querying system statistics using built-in, secure native channels (SSH/WinRM) rather than installing permanent background software.	Maintains zero footprint on the client OS; eliminates massive security compliance hurdles.
Micro-Idle Window	Highly granular 15-minute scanning cycles designed to catch mid-day developer inactivity and short-term resource abandonment.	Catches immediate fiscal waste that standard hourly/daily cloud cost trackers completely miss.
FinOps Ontology	A system map that knows exactly which instance belongs to which developer, what its budget limit is, and what its billing tier costs.	Allows the AI to explain the context of why an instance was turned off and precisely who it impacted.
Cold Start Latency	The 1-to-2-minute initialization penalty a virtual machine incurs when booting its kernel, OS, and web servers from a stopped state.	The core technical limitation we overcome using Predictive Pre-Warming and JIT Lambda routers.

Instance Morphing	Dynamically modifying an active server's type profile (e.g., changing from an expensive instance to an ultra-cheap size) rather than powering it down.	Our primary mitigation strategy for single-instance web applications; cuts cost by 90% while keeping the IP address live.
Guardian Validator	An automated safety-net daemon that audits system states for active connections before allowing an optimization script to turn off a machine.	Ensures absolute safety by design; prevents automated logic from ever causing accidental business downtime.

Technology which used:

Layer	Component	Technology Chosen	Specific Purpose in OptiScale
Presentation	User Interface	Next.js + TypeScript	Sleek, dark-mode command cockpit with high-density tracking data.

Presentation	Real-Time Streams	WebSockets (Client API)	Feeds live instance status badges and grace-period countdown bars.
Backend API	Core Logic Gate	FastAPI (Python)	Handles async loops, processes webhooks, and serves database queries.
Cloud Control	Infrastructure SDK	Boto3 (AWS SDK)	Programmatically executes server power states, tagging, and sizing adjustments.
Agentless Scan	OS-Level Access	Paramiko (Python SSH)	Securely logs into Linux VM kernels to run raw socket inspection commands.
Data & Storage	Persistent DB Map	PostgreSQL	Stores user roles, centralized operational schedules, and override blocks.
Data & Storage	Analytics Engine	TimescaleDB	Specialized PostgreSQL engine to handle large time-series cost history data.
Serverless	Just-In-Time Router	AWS Lambda	Instantly executes low-latency "Fast-Boot" triggers when users request access.

Governance	Enterprise Sync	Jira REST Webhooks	Connects automated infrastructure actions directly to human approval workflows.
Infrastructure	Containerization	Docker & Docker Compose	Packages the platform plane into reproducible, isolated, zero-setup environments.

OptiScale vs Middleware.io

Why OptiScale is a Closed-Loop Automation Control Plane, not just an Observability Tool.

#	The Challenge	What Middleware Cannot Do (Limit)	How OptiScale Solves It
①	The "Stateful" OS-Level Session Lock	- Middleware sees low CPU/RAM and may shutdown server even if developer is logged in via RDP/SSH. ⇒ Active human sessions can get dropped.	⇒ OptiScale checks OS socket layers (/proc/net/tcp or quser on Windows) to detect active user sessions. If active session exists, shutdown is overridden.
②	Just-In-Time (JIT) "Fast-Boot" for Non-Admins	Middleware is for DevOps/SRE teams. No self-service portal for non-admin users (QA, developers) to start stopped resources safely.	⇒ OptiScale Command Center provides self-service dashboard. AWS Lambda validates identity, checks schedule DB and wakes up resources instantly.
③	Native JIRA / ServiceNow Approval State-Machines	Middleware can raise alerts or create tickets, but cannot wait for actual approval in JIRA / ServiceNow and then resume operations.	⇒ OptiScale creates bi-directional webhook loop with JIRA / ServiceNow. Pauses execution, waits for "Approved" callback via APIs / WebSockets and then proceeds automatically.
④	Handling Legacy, Non-Containerized Environments (e.g., SAP)	APM tools rely on containers, eBPF, agents which are complex and brittle on legacy systems like Windows Servers / SAP.	⇒ OptiScale is agentless. Uses native OS tools (Windows Task Scheduler, Linux Cron) that exist since 1990s. Easy, non-invasive, enterprise-compliant.
⑤	Dynamic IP Address Flapping Prevention	Middleware observes IP changes but does not manage networking. When instance stops, IP changes and breaks domain mapping.	⇒ OptiScale integrates AWS Elastic IP (EIP) preservation in Boto3 engine. Locks and remaps static IP during start/stop cycles. Zero-error mapping.
⑥	The "Micro-Idle" 15-Minute Audit Window	Enterprise APM tools aggregate metrics hourly/daily. Cannot catch short idle periods (like 2-hr lunch break).	⇒ OptiScale runs 15-minute auditing daemon with 10-minute grace period. Detects & stops micro-idle resources that others ignore.
⑦	Cost-per-Request Unit Economics Scaling	Middleware shows total cost and performance, but cannot calculate cost-per-transaction for a microservice.	⇒ OptiScale combines AWS billing snapshots + API traffic telemetry to compute exact cost-per-request. Can throttle / alert when uneconomical.
⑧	Budget-Cap Proactive Auto-Remediation	When budget is breached, Middleware sends email alerts. Expensive servers keep running until human action.	⇒ OptiScale shifts to Auto-Remediation Mode. Executes policy-based downgrades / shutdowns automatically via cloud APIs. Stops financial bleeding.
⑨	Predictively Pre-Warming "Cold Start" Environments	Observability tools cannot see stopped resources. Cannot predict when to start them. User faces cold-start delays.	⇒ OptiScale uses historical usage data in PostgreSQL schedule DB. Predictive pre-warm (e.g., 7:55 AM for 8:00 AM usage). No cold-start.
⑩	Managing Hard "Leaves" & Holiday Schedulers	Middleware has no concept of HR calendar, leaves, holidays. Sandbox instances stay running unnecessarily.	⇒ OptiScale integrates team calendars & JIRA leave data. Automatically deep-sleeps resources during leaves / holidays.

★ ULTIMATE PITCH POINT:

We are not trying to replace Middleware's powerful observability & visualization. We are building the missing OPERATIONAL EXECUTION ARM that bridges enterprise business rules with automated cloud infrastructure controls.

OptiScale = Observe + Decide + Act + Optimize (Closed-Loop)